

VIXOR Signal Runtime Contract Audit

Task 1.1 — READ-ONLY ARCHITECTURAL AUDIT

Date: 2026-08-09

Classification: Confidential — Internal

Document Version: 1.0

Table of Contents

Table of Contents

2

1. Executive Summary

5

2. Current Architecture

5

2.1 End-to-End Data Flow

5

2.2 Key Files

6

2.3 Data Flow Summary

6

3. Contract Conflicts

6

3.1 Conflict #1: Terminal Status Disagreement (tp1_hit / tp2_hit)

6

3.2 Conflict #2: Invalidated Status Not in Database

7

3.3 Conflict #3: evaluateTrackingPrice Does Not Handle tp1_hit/tp2_hit

7

3.4 Conflict #4: resolved_at Set on Non-Resolved Signals

7

3.5 Conflict #5: Event Name Mismatch Between Engine and Orchestrator

7

3.6 Conflict #6: Cancel Bypasses Engine

8

3.7 Conflict #7: Expiration Bypasses Engine

8

3.8 Conflict Summary

8

4. Security Findings

8

4.1 Attack Path Matrix

9

4.2 Authority Model Comparison

9

5. State Matrix

10

5.1 Complete State Treatment Matrix

10

5.2 Terminal State Comparison

10

6. State vs hitTp Consistency Matrix

11

6.1 Valid State + hitTp Combinations

11

6.2 Invalid State + hitTp Combinations

11

6.3 Engine hitTp Semantics

12

7. DB Compatibility Findings

12

7.1 Database Enum Definition

12

7.2 Required Schema Change	12
8. Non-Price Transition Findings	13
8.1 Engine Non-Price Policy	13
8.2 Current Callers	13
8.3 Business Policy Gap	13
9. Timestamp / Observation Findings	13
9.1 Engine observedAt Validation	14
9.2 Current Runtime Timestamp Behavior	14
9.3 Out-of-Order Observations	14
10. TP Crossing Findings	14
10.1 Current Behavior	14
10.2 Tradeoffs	15
11. Event Findings	15
11.1 Engine Event Types	15
11.2 Orchestrator Event Names	15
11.3 Actual Notification Path	16
11.4 MOXI Event Consumers	16
12. Recommended Contract	16
12.1 Authoritative State (A)	16
12.2 Expected TP Derivation (B)	16
12.3 Contract Items C-N	17
13. Exact Files That Will Need Changes (for Task 2+)	17
14. P0 / P1 / P2 Classification	18
14.1 P0 — Must Fix Before Task 2	18
14.2 P1 — Must Fix Before Live	19
14.3 P2 — Defer	19
15. Risks	19
16. Open Decisions	19
17. Proposed Task 2 Scope	20

17.1 Task 2 Deliverables 20

17.2 Out of Scope for Task 2 20

1. Executive Summary

This document presents the findings of a read-only architectural audit of the VIXOR Signal Runtime Contract system, conducted as part of Task 1.1. The audit examined the current production code path for signal tracking, the recently introduced Transition Engine, and the contracts (implicit and explicit) between all system components. The goal was to identify contract conflicts, security vulnerabilities, and architectural gaps before any code changes are made in subsequent tasks. The audit scope covers all TypeScript source files related to signal tracking, the database schema, generated Supabase types, the event orchestrator, and the MOXI notification and context engine components.

The most critical finding is that the Transition Engine, while architecturally sound and representing the correct long-term design, is currently a pure domain authority with **zero runtime authority**. The production code path does not call the engine at all. Instead, the client-side `useSignalMonitor` hook evaluates price transitions locally and sends pre-computed status and `hitTp` values to `updateSignalTracking()`, which writes them directly to the database with no validation against the engine or the current DB state. The client currently has full authority over signal status transitions, a fundamental security and correctness concern that must be resolved before any further development.

CRITICAL FINDING: Engine Disconnected from Runtime

`updateSignalTracking()` accepts client-provided status and `hitTp` and writes directly to DB. The engine has zero runtime authority. 7 contract conflicts, 10 attack paths, DB schema mismatch for invalidated.

Severity	Count	Description	Resolution Target
P0 (Critical)	6	Must fix before Task 2	Task 2 or earlier
P1 (High)	2	Must fix before live deployment	Before go-live
P2 (Medium)	1	Defer to appropriate task	Task 4+

The audit revealed the codebase operates as two parallel systems that do not communicate: the legacy runtime (`functions.ts` + `use-signal-monitor.ts`) and the new Transition Engine (`transition-engine.ts`). Until unified, the engine serves only as documentation of intended design, not enforcement. All 7 contract conflicts stem from this disconnect. The 10 attack paths in Section 4 are all exploitable because server-side validation is absent. This report provides complete analysis and a recommended contract (Section 12) that resolves all identified issues.

2. Current Architecture

2.1 End-to-End Data Flow

- User clicks Track → `createSignalTracking()` writes a new row to DB with `status=pending`, `hitTp=0`.
- `useSignalMonitor` polls `useLivePrices` WebSocket for real-time price updates on the tracked symbol.
- On each price tick, `evaluateTrackingPrice()` in `functions.ts` (NOT `transition-engine.ts`) computes `newStatus` and `hitTp` on the CLIENT side.
- Client sends `{trackingId, status, currentPrice, hitTp}` to `updateSignalTracking()` server function.
- Server writes `status`, `hitTp`, `current_price` directly to DB. Server also sends notification via `notificationRouter`.

The critical architectural observation is that price evaluation logic lives in `functions.ts`, not in `transition-engine.ts`. The Transition Engine was built to be the authoritative state machine, but it is not wired into the runtime path at all. The client-side monitor makes all transition decisions independently, and the server acts as a passive write-through layer. This

means the engine and the runtime are two separate parallel systems that do not communicate with each other, creating the conditions for all 7 contract conflicts documented in Section 3.

2.2 Key Files

File	Role	Status
transition-engine.ts	Defines price/non-price transitions, event types, terminal states	DEFINED but NOT used in runtime
functions.ts	Contains <code>evaluateTrackingPrice()</code> + <code>updateSignalTracking()</code> — the ACTUAL runtime path	ACTIVE in production
use-signal-monitor.ts	Client-side hook — polls prices, calls <code>evaluateTrackingPrice</code> , sends updates	ACTIVE in production
types.ts	SignalStatus type + TERMINAL_STATUSES constant	ACTIVE — conflicts with engine

2.3 Data Flow Summary

Step	Component	Action	Authority
1	Client	User clicks Track button	User-initiated
2	Server: <code>createSignalTracking</code>	Writes pending row to <code>signal_tracking</code>	Server writes
3	Client: <code>useSignalMonitor</code>	Starts polling prices via WebSocket	Client polls
4	Client: <code>evaluateTrackingPrice</code>	Computes <code>newStatus</code> + <code>hitTp</code> from price	CLIENT decides
5	Server: <code>updateSignalTracking</code>	Writes client-provided <code>status/hitTp</code> to DB	Server pass-through
6	Server: <code>notificationRouter</code>	Sends notification based on status	Server acts

The authority model is clear from this table: the client decides the status transition (step 4), and the server merely records it (step 5). The Transition Engine, which should own step 4, is not in the path. This is the root cause of every finding in this report. The server has become a passive write layer that trusts the client completely. No state validation, no transition legality check, no engine consultation occurs at any point in this flow.

3. Contract Conflicts

This section documents all 7 contract conflicts discovered between the Transition Engine, the production runtime, the TypeScript type system, and the database schema. Each conflict represents a place where two or more components disagree about the fundamental behavior of the signal tracking system. These disagreements range from which states are terminal, to which function handles price evaluation, to what event names are used.

3.1 Conflict #1: Terminal Status Disagreement (`tp1_hit` / `tp2_hit`)

CONFLICT #1: tp1_hit/tp2_hit Terminal Status

TERMINAL_STATUSES in types.ts includes tp1_hit and tp2_hit as terminal. TRANSITION_TERMINAL_STATUSES in transition-engine.ts does NOT include them (they are intermediate). useSignalMonitor stops monitoring when it sees a terminal status, so if TP1 is hit, monitoring stops — but the engine says it should continue for TP2/SL.

The TERMINAL_STATUSES array in types.ts is defined as: [tp1_hit, tp2_hit, tp3_hit, sl_hit, expired, cancelled, invalidated]. This is used by useSignalMonitor to decide whether to continue monitoring a signal. When a signal reaches tp1_hit, the monitor sees it as terminal and stops. But the Transition Engine defines TRANSITION_TERMINAL_STATUSES as: [tp3_hit, sl_hit, expired, cancelled, invalidated] — notably excluding tp1_hit and tp2_hit, which are intermediate states in the engine's model. This means the engine expects monitoring to continue past TP1 hits to check for TP2 and SL, but the monitor has already stopped. For multi-TP signals, this is a functional correctness bug.

3.2 Conflict #2: Invalidated Status Not in Database

The SignalStatus TypeScript type includes 'invalidated' as a valid status value, giving the domain model 9 possible states. However, the database enum (migration 20260629000000) only defines 8 values: pending, active, tp1_hit, tp2_hit, tp3_hit, sl_hit, expired, cancelled. The 'invalidated' value is missing from the database. The generated Supabase types.ts (line 1626-1635) confirms only 8 values. If any code path attempts to write status='invalidated' to the database, it will fail with a Postgres enum constraint violation.

3.3 Conflict #3: evaluateTrackingPrice Does Not Handle tp1_hit/tp2_hit

The evaluateTrackingPrice() function in functions.ts only handles status 'pending' and 'active'. When the current status is tp1_hit or tp2_hit, the function returns 'none' (no transition), regardless of the price. Even if the monitor continued past tp1_hit (which it does not due to Conflict #1), the price evaluation function would be unable to process any further transitions. This creates a double barrier: the monitor stops monitoring, and even if it didn't, the evaluation function wouldn't evaluate. The Transition Engine handles all non-terminal states in its PRICE_TRANSITION_MATRIX, including tp1_hit and tp2_hit.

3.4 Conflict #4: resolved_at Set on Non-Resolved Signals

The server function updateSignalTracking() uses TERMINAL_STATUSES (from types.ts) to decide whether to set the resolved_at timestamp and send notifications. Since tp1_hit and tp2_hit are in this terminal list, resolved_at gets set when TP1 is hit — but the signal is NOT actually resolved. The signal should continue being tracked for TP2 and SL. Setting resolved_at on a non-resolved signal creates a semantic contradiction that downstream consumers (analytics, reporting) would misinterpret.

3.5 Conflict #5: Event Name Mismatch Between Engine and Orchestrator

Engine Event (transition-engine.ts)	Orchestrator Event (orchestrator.ts)	Match?
ENTRY_REACHED	signal.tracking.created	NO
TP1_HIT, TP2_HIT, TP3_HIT	signal.tp_hit (singular)	NO — different granularity
SL_HIT	signal.sl_hit	Conceptual match, different name
SIGNAL_INVALIDATED	(not defined)	MISSING in orchestrator
SIGNAL_EXPIRED	signal.expired	Conceptual match, different name

Engine Event (transition-engine.ts)	Orchestrator Event (orchestrator.ts)	Match?
SIGNAL_CANCELLED	(not defined)	MISSING in orchestrator

3.6 Conflict #6: Cancel Bypasses Engine

`cancelSignalTracking()` in `functions.ts` writes `status='cancelled'` directly to the database without consulting the Transition Engine. The engine defines a `NON_PRICE_TRANSITIONS` map that specifies which non-price transitions are allowed from each state. The cancel function does not check this map and does not verify the current state is non-terminal. It could cancel an already-terminal signal (e.g., one that already hit `tp3_hit`), creating an inconsistent database record. The engine would deny this transition, but the server never asks.

3.7 Conflict #7: Expiration Bypasses Engine

`useSignalMonitor`'s expiration logic writes `status='expired'` directly via `updateSignalTracking` without engine consultation. While the monitor does check `TERMINAL_STATUSES.includes(t.status)` before expiring, it uses the OLD terminal list that includes `tp1_hit/tp2_hit` as terminal — meaning it would NOT expire a signal in `tp1_hit` state, even though the engine says `tp1_hit` is non-terminal and SHOULD be eligible for expiration. This creates inconsistent expiration behavior depending on which component you ask.

3.8 Conflict Summary

#	Conflict	Components	Impact
1	Terminal status: <code>tp1_hit/tp2_hit</code>	<code>types.ts</code> vs engine	Monitor stops too early
2	Invalidated not in DB	<code>types.ts</code> vs DB schema	Potential runtime crash
3	<code>evaluateTrackingPrice</code> gaps	<code>functions.ts</code> vs engine	No multi-TP progression
4	resolved_at on non-resolved	<code>functions.ts</code> vs domain semantics	Analytics corruption
5	Event name mismatch	engine vs orchestrator	Integration failure
6	Cancel bypasses engine	<code>functions.ts</code> vs engine	Illegal transitions
7	Expiration bypasses engine	monitor vs engine	Inconsistent expiration

4. Security Findings

This section documents the security posture of the current signal tracking runtime. The central finding is that the client has unrestricted authority over signal status transitions. The server function `updateSignalTracking()` accepts a request containing `trackingId`, `status`, `currentPrice`, and `hitTp` from the client, and writes them directly to the database. The validation is minimal: it checks that `trackingId` and `status` are non-empty strings. There is NO validation that the status transition is legal, NO check against the current DB state, NO consultation of the Transition Engine, and NO verification that the provided price is consistent with the claimed status. This section enumerates 10 conceptual attack paths that are possible under the current architecture.

CRITICAL: Client Has Full Status Authority

`updateSignalTracking()` accepts `{trackingId, status, currentPrice, hitTp}` from the client and writes them directly. The validator only checks non-empty strings. No transition legality check, no DB state check, no engine consultation. A client can send `status='hacked'` and it would be written to the database.

4.1 Attack Path Matrix

#	Attack	From	To	Engine Would Allow?	Current Risk
1	Skip to final TP	pending	tp3_hit	NO	EXPLOITABLE
2	Skip to final TP	active	tp3_hit	NO	EXPLOITABLE
3	Premature SL trigger	active	sl_hit	NO (needs price)	EXPLOITABLE
4	Claim TP1 without price	active	tp1_hit	NO (needs price >= TP1)	EXPLOITABLE
5	Foreign trackingId	any	any	N/A	PARTIALLY MITIGATED (RLS)
6	Stale tracking update	any	any	N/A	EXPLOITABLE (no version)
7	Replayed update	any	any	N/A	EXPLOITABLE (no idempotency)
8	Duplicate update	any	any	N/A	EXPLOITABLE
9	Invalid status enum	any	'hacked'	NO	EXPLOITABLE (TS types erased)
10	Forged hitTp	active	active + hitTp=3	INCONSISTENT	EXPLOITABLE

Attack paths 1-4 are the most dangerous because they allow the client to fabricate signal outcomes. Path 1 (pending to tp3_hit) lets a client claim all three take-profit levels were hit simultaneously, triggering false notifications and corrupting analytics. Path 9 is particularly insidious: TypeScript type annotations are erased at runtime, so the validator cannot distinguish between a legitimate status value and an arbitrary string. The validator casts the unknown input to the expected type shape without checking enum membership. A client sending `status='hacked'` would pass validation and be written to the database.

4.2 Authority Model Comparison

Aspect	Current (Insecure)	Expected (Secure)
Who decides status?	CLIENT	SERVER (via engine)
Client sends	<code>{trackingId, status, hitTp, price}</code>	<code>{trackingId, observedPrice, observedAt}</code>
Server reads current state?	NO	YES — reads from DB
Server calls engine?	NO	YES — <code>evaluateSignalTransition()</code>
Server validates transition?	NO	YES — engine denies illegal transitions
Server derives hitTp?	NO — trusts client	YES — derives from status

Aspect	Current (Insecure)	Expected (Secure)
Idempotency?	NO	YES — (trackingId, observedAt, price)
Optimistic concurrency?	NO	YES — version/updated_at check

The gap between the current and expected models is the primary deliverable of Task 2. Every row in this table represents a code change that must be made before the signal tracking system can be considered production-secure. The Transition Engine already implements the server-side logic for the expected model; the task is to wire it into the runtime path and remove client authority over status and hitTp.

5. State Matrix

This section presents a comprehensive matrix showing how each possible signal status is treated across all five relevant components of the system. This matrix makes every disagreement immediately visible and serves as the definitive reference for understanding the scope of the contract conflicts. Each row represents one status value, and each column represents how one component classifies or handles that status. Disagreements between columns indicate contract conflicts.

5.1 Complete State Treatment Matrix

State	Engine	Types	Client Monitor	Server Functions	DB Enum
pending	NON-TERMINAL	NON-TERMINAL	NON-TERMINAL	NON-TERMINAL	IN DB
active	NON-TERMINAL	NON-TERMINAL	NON-TERMINAL	NON-TERMINAL	IN DB
tp1_hit	INTERMEDIATE	TERMINAL	TERMINAL (stops)	NOT HANDLED	IN DB
tp2_hit	INTERMEDIATE	TERMINAL	TERMINAL (stops)	NOT HANDLED	IN DB
tp3_hit	TERMINAL	TERMINAL	TERMINAL	TERMINAL	IN DB
sl_hit	TERMINAL	TERMINAL	TERMINAL	TERMINAL	IN DB
invalidated	TERMINAL	IN TYPES (not terminal list)	N/A	N/A	NOT IN DB
expired	TERMINAL	TERMINAL	TERMINAL	TERMINAL	IN DB
cancelled	TERMINAL	TERMINAL	TERMINAL	TERMINAL	IN DB

Key observations from this matrix: (1) The tp1_hit and tp2_hit rows show a three-way disagreement between the engine (INTERMEDIATE), types.ts (TERMINAL), and functions.ts (NOT HANDLED). This is the most impactful conflict because it affects multi-TP signals directly. (2) The invalidated row shows it is TERMINAL in the engine, defined in types.ts but NOT in the terminal list, and completely absent from the database. This is a schema-level gap. (3) All other states (pending, active, tp3_hit, sl_hit, expired, cancelled) are consistent across all components.

5.2 Terminal State Comparison

Component	Terminal States	Count
Engine (TRANSITION_TERMINAL_STATUSES)	tp3_hit, sl_hit, expired, cancelled, invalidated	5
Types (TERMINAL_STATUSES)	tp1_hit, tp2_hit, tp3_hit, sl_hit, expired, cancelled, invalidated	7
DB Enum	(no explicit terminal concept — all 8 are valid values)	N/A

The difference between 5 and 7 terminal states is exactly the two states tp1_hit and tp2_hit. The engine says these are intermediate, the types say they are terminal. This two-state difference is the single most impactful conflict in the entire audit because it determines whether multi-TP signals work correctly. Resolving this conflict (removing tp1_hit and tp2_hit from TERMINAL_STATUSES in types.ts) is classified as P0 and must be done before Task 2.

6. State vs hitTp Consistency Matrix

This section examines the relationship between the status field and the hitTp field. The hitTp field records which take-profit level was most recently hit (0=none, 1=TP1, 2=TP2, 3=TP3). Ideally, status and hitTp should always be consistent: if status=tp1_hit then hitTp should be 1. This section documents which combinations are valid, which are invalid, and what happens when inconsistent combinations are encountered by the Transition Engine.

6.1 Valid State + hitTp Combinations

Status	hitTp	Assessment	Notes
pending	0	VALID EXPECTED	No TP hit yet
active	0	VALID EXPECTED	Entry reached, no TP hit
tp1_hit	1	VALID EXPECTED	First TP level hit
tp2_hit	2	VALID EXPECTED	Second TP level hit
tp3_hit	3	VALID EXPECTED	Final TP level hit

6.2 Invalid State + hitTp Combinations

Status	hitTp	Assessment	Problem
active	1	INCONSISTENT	If TP1 hit, status should be tp1_hit
active	2	INCONSISTENT	Jumped — status says active but hitTp says TP2
active	3	INCONSISTENT	Jumped — status says active but hitTp says TP3
tp1_hit	0	INCONSISTENT	Status says TP1 hit but hitTp says none
tp1_hit	2	INCONSISTENT	Jumped — TP2 without TP1
tp2_hit	0	INCONSISTENT	Status says TP2 hit but hitTp says none
tp2_hit	3	INCONSISTENT	Jumped — TP3 without TP2
tp3_hit	2	INCONSISTENT	Status says TP3 but hitTp says TP2

6.3 Engine hitTp Semantics

The engine uses hitTp as the nextTpIndex (line 390 of transition-engine.ts: `const nextTpIndex = hitTp`). This means hitTp tells the engine which TP level to evaluate NEXT, not which was hit last. For active + hitTp=0, the engine checks TP1 (index 0). If TP1 is hit (`price >= TP1`), it returns `tp1_hit` and sets `hitTp=1` in the result. But if a forged hitTp=1 is provided with `currentState=active`, the engine would check TP2 (index 1) instead of TP1 (index 0). This means a forged hitTp doesn't just create inconsistency — it changes which TP level the engine evaluates, potentially skipping levels entirely. No validation exists to check that hitTp is consistent with the current status. The engine assumes the caller provides a consistent hitTp value. Legacy DB rows may contain inconsistent combinations from the old system, and the engine would process them incorrectly.

WARNING: hitTp Forging Shifts TP Evaluation

The engine derives the next TP to check from the caller-provided hitTp value. A forged or stale hitTp shifts evaluation to the wrong TP level, potentially skipping take-profit targets. No pre-check validates state/hitTp consistency on engine entry.

7. DB Compatibility Findings

This section documents the exact state of the database schema as it relates to signal status values, the generated Supabase types, and the gap between the database and the TypeScript domain model. The analysis is based on the migration file 20260629000000 and the auto-generated types.ts from Supabase.

7.1 Database Enum Definition

Source	Values	Count
DB Migration (20260629000000)	pending, active, tp1_hit, tp2_hit, tp3_hit, sl_hit, expired, cancelled	8
Generated Supabase types.ts (line 1626-1635)	pending, active, tp1_hit, tp2_hit, tp3_hit, sl_hit, expired, cancelled	8
TypeScript SignalStatus type (types.ts)	+ invalidated	9

The database and generated types are in agreement at 8 values. The TypeScript domain type adds one value: invalidated. This creates a one-directional type safety gap. TypeScript code that uses the SignalStatus type can reference invalidated, and the TypeScript compiler will not flag it as an error. But if that value is ever written to the database, Postgres will reject it with an enum constraint violation. The error would only surface at runtime.

DB SCHEMA GAP: invalidated Missing

The TypeScript domain type includes 'invalidated' but the DB enum does not. If any code path attempts to write `status='invalidated'` to the database, it will fail with a Postgres enum constraint violation. Currently no code writes it (engine is disconnected), but connecting the engine in Task 2 would trigger this.

7.2 Required Schema Change

If invalidated is to become a valid tracking state, the following migration is required:

```
ALTER TYPE signal_status ADD VALUE 'invalidated';
```

However, several design decisions must be made before adding this value: (1) Is invalidated a signal-level concept or a tracking-level concept? A signal might be invalidated (e.g., the analysis was wrong) but the tracking might have already partially completed. (2) Does invalidated require additional columns, such as an `invalidation_reason` field? (3) Should invalidated trigger notifications like other terminal states? The Transition Engine already produces transition decisions with `to='invalidated'`, so the engine is ready for this status. But since the engine is not connected to the runtime, this cannot currently happen in production. The schema change should be made in Task 1.2 after these business decisions are documented.

8. Non-Price Transition Findings

The Transition Engine allows three non-price transitions from any non-terminal state: cancelled, expired, and invalidated. This section examines how these transitions are currently handled in the production runtime, and identifies gaps between the engine's policy and the actual behavior.

8.1 Engine Non-Price Policy

The engine defines `NON_PRICE_TRANSITIONS` as a map from `SignalStatus` to the set of allowed non-price transitions. The current policy is permissive: pending, active, `tp1_hit`, and `tp2_hit` can all transition to cancelled, expired, or invalidated. Terminal states (`tp3_hit`, `sl_hit`, expired, cancelled, invalidated) have no allowed non-price transitions. This is a generalization of the original implementation brief, which was more restrictive about which specific transitions were allowed from which states.

8.2 Current Callers

Caller	Transition	Engine Consulted?	State Check?	Issue
<code>cancelSignalTracking()</code>	→ cancelled	NO	NO	No check on current state; could cancel terminal signal
<code>useSignalMonitorExpiration</code>	→ expired	NO	YES (but wrong list)	Uses TERMINAL_STATUSES that includes <code>tp1_hit/tp2_hit</code>
No code path	→ invalidated	N/A	N/A	Only defined in engine, never produced in runtime

8.3 Business Policy Gap

The engine's permissive policy raises an important business question: should cancellation be allowed after a partial TP hit? For example, if TP1 was hit (the user partially profited), cancelling the tracking would lose the TP1 result and could mislead analytics. The engine allows `tp1_hit` → cancelled and `tp2_hit` → cancelled, but the business implications have not been documented. Before Task 2, the explicit business policy for each non-price transition from each state must be documented. This includes whether `resolved_at` should be set for partial cancellations, whether notifications should fire, and whether analytics should count a partially-hit signal as a win, loss, or separate category.

9. Timestamp / Observation Findings

This section examines how timestamps are handled in the signal tracking system, including the engine's `observedAt` field, the server's timestamp generation, and the absence of market data timestamps. Proper timestamp handling is essential for

audit trails, replay protection, and analytics accuracy.

9.1 Engine observedAt Validation

The Transition Engine defines `observedAt` as an ISO 8601 timestamp of the price observation. Validation is performed by `isValidDateString()`, which only checks that `new Date(str)` is not NaN. This accepts any valid JavaScript date string, not strict ISO 8601. For example, 'Aug 9 2026' would pass validation but is not ISO 8601. The function does not enforce the T separator, timezone specification, or millisecond precision. For a financial system where observation timing is critical, this loose validation is insufficient.

9.2 Current Runtime Timestamp Behavior

Timestamp	Source	Format	Recorded Where?
<code>created_at</code>	DB default	Postgres timestampz	signal_tracking table
<code>updated_at</code>	Server: <code>new Date().toISOString()</code>	ISO 8601	signal_tracking table
<code>resolved_at</code>	Server: <code>new Date().toISOString()</code>	ISO 8601	signal_tracking table (terminal only)
<code>observedAt</code>	Engine defines it	ISO 8601 (intended)	NOT passed by client, NOT recorded
<code>serverReceivedAt</code>	Not defined	N/A	NOT recorded
<code>marketDataTimestamp</code>	Not defined	N/A	NOT recorded

9.3 Out-of-Order Observations

The production price source is a WebSocket (`useLivePrices`). WebSocket messages can arrive out of order during reconnection or network issues. The `useSignalMonitor` hook uses a `previousPriceRef` to skip duplicate prices, but this only prevents redundant local evaluations. It does not detect stale or out-of-order observations from the price source. There is no sequence number, no timestamp ordering, and no server-side mechanism to reject late-arriving observations. In a high-volatility market, an out-of-order observation could cause the system to process a stale price after a newer one, potentially triggering incorrect transitions or missing real ones.

WARNING: No Observation Timestamp Chain

No `serverReceivedAt`, no `marketDataTimestamp`, no observation ordering, no sequence numbers. The server records when IT processed the update, but never when the price was actually observed on the market. WebSocket messages can arrive out of order during reconnection. No mechanism detects or rejects stale observations.

10. TP Crossing Findings

This section documents how the system handles price jumps that cross multiple take-profit levels in a single observation. This is a common scenario in volatile markets where price gaps can skip one or more TP levels between consecutive observations.

10.1 Current Behavior

When price jumps across multiple TP levels (e.g., from 109 to 125 where TP1=110, TP2=120, TP3=130), the system produces only ONE transition: to TP1 (tp1_hit). This is because both evaluateTrackingPrice() and the Transition Engine evaluate sequentially. The engine checks the next TP based on hitTp. For active + hitTp=0, it only checks TP1 (index 0). If TP1 is hit (price >= 110), it returns tp1_hit and stops. It does NOT continue to check TP2. This is consistent with a 'one transition per observation' policy.

10.2 Tradeoffs

Approach	Pros	Cons	Recommendation
One transition per observation	Simple, conservative, no reentrancy	May miss actual TP hits in gaps	RECOMMEND for live ticks
Multiple transitions per observation	Accurate, captures all TP hits	Reentrancy, event ordering, complexity	Defer decision
Worst-case evaluation	Catches SL before TP on gaps	More complex, still single output	Engine has OHLC comment only

For live tick-by-tick data, one transition per observation is reasonable because ticks arrive frequently and subsequent ticks will catch the next TP. For OHLC candle data where each observation represents a time range (e.g., 15 minutes), a single candle could span all three TPs. The engine has an 'OHLC Ambiguity Policy' comment suggesting worst-case ordering (check SL before TP for BUY signals), but the multi-TP crossing policy for OHLC is undefined. The current unit tests confirm the one-transition behavior (test: 'price jumping past TP1 only triggers TP1'). This MUST be explicitly decided and documented before live deployment.

11. Event Findings

This section examines the event systems in the codebase. The Transition Engine defines 8 event types. The Event Orchestrator defines different event names. These are completely separate event systems with no bridge between them. Critically, no code currently emits any signal-related events at all.

11.1 Engine Event Types

Engine Event	Description	Status
ENTRY_REACHED	Price crossed entry level	Defined, never emitted
TP1_HIT	Price crossed TP1 level	Defined, never emitted
TP2_HIT	Price crossed TP2 level	Defined, never emitted
TP3_HIT	Price crossed TP3 level	Defined, never emitted
SL_HIT	Price crossed stop-loss level	Defined, never emitted
SIGNAL_INVALIDATED	Signal was invalidated	Defined, never emitted
SIGNAL_EXPIRED	Signal expired without hitting TP/SL	Defined, never emitted
SIGNAL_CANCELLED	Signal was cancelled by user	Defined, never emitted

11.2 Orchestrator Event Names

Orchestrator Event	Scope	Handler Registered?
signal.tracking.created	Per tracking	NO
signal.generated	Per signal	NO
signal.expired	Per signal	NO
signal.tp_hit	Per tracking (no TP level)	NO
signal.sl_hit	Per tracking	NO

11.3 Actual Notification Path

The `updateSignalTracking()` server function sends notifications directly via `notificationRouter`, completely bypassing both the engine events and the orchestrator event bus. A `grep` confirms that NO code currently calls `VixorEvents.emit()` for any signal-related event. The orchestrator is a typed event bus with zero registered handlers for signal events. The duplication between `signal.tp_hit` (orchestrator, singular) and `TP1_HIT/TP2_HIT/TP3_HIT` (engine, per-level) represents a granularity mismatch that must be resolved when unifying the event vocabulary in Task 4.

11.4 MOXI Event Consumers

MOXI `notification-hub.ts` calls `detectSignalProximity()` but reads from DB, not from events. MOXI `context-engine.ts` reads `activeTrackings` from DB with `.in('status', ['pending', 'active'])`, which means it already misses `tp1_hit/tp2_hit` signals due to the terminal status conflict (Conflict #1). Both MOXI components are DB-polling consumers, not event-driven consumers. They are affected by the terminal status conflict because their status filter excludes intermediate states that the engine says should still be active.

DEAD CODE: Zero Event Emissions

No code emits any signal-related events. The engine defines 8 event types that are never emitted. The orchestrator defines 5 event names with zero registered handlers. The server sends notifications directly via `notificationRouter`, bypassing both event systems entirely. All event infrastructure is dead code.

12. Recommended Contract

Based on all findings in Sections 1-11, this section defines the recommended Signal Runtime Contract. This contract represents the target architecture that subsequent tasks will implement. Each item is labeled with a letter for reference in Task 2 scope (Section 17).

12.1 Authoritative State (A)

The status column in the `signal_tracking` table is the source of truth. It can only be changed by the server calling `evaluateSignalTransition()`. No client, no direct DB write, no other server function may modify status. The engine is the sole authority for determining whether a transition is allowed.

12.2 Expected TP Derivation (B)

Current Status	Expected hitTp	Notes
pending	0	No TP hit yet
active	0	Entry reached, no TP
tp1_hit	1	First TP hit
tp2_hit	2	Second TP hit
tp3_hit	3	Terminal, N/A

The expected hitTp should be DERIVED from currentState, not provided by the caller. The server computes it from status on every write. This eliminates an entire class of inconsistency bugs and removes hitTp from the client input surface.

12.3 Contract Items C-N

Item	Rule	Rationale
(C) hitTp Semantics	Derived consistency metadata, NOT client input	Eliminates forging
(D) Terminal States	Only tp3_hit, sl_hit, invalidated, expired, cancelled	tp1_hit/tp2_hit are INTERMEDIATE
(E) Legal Transitions	Engine PRICE_TRANSITION_MATRIX + NON_PRICE_TRANSITIONS	Engine is authority
(F) Non-Price	Cancel/expire from any non-terminal; invalidate deferred	Business policy needed
(G) Invalidated	Defer to Task 1.2	Signal vs tracking level TBD
(H) Timestamps	Strict ISO 8601 observedAt + serverReceivedAt + market data ts	Audit trail
(I) TP Crossing	One-transition-per-observation for live; defer OHLC	Conservative default
(J) Events	Engine event types = canonical vocabulary	Unify orchestrator
(K) Client Input	{trackingId, observedPrice, observedAt} only	Minimal surface
(L) Server Authority	Read DB state → call engine → commit if allowed	Full validation
(M) Concurrency	Optimistic: version column or updated_at comparison	Prevent lost updates
(N) Idempotency	Key = (trackingId, observedAt, observedPrice)	Safe retries

13. Exact Files That Will Need Changes (for Task 2+)

This section provides a file-level change inventory for all tasks that follow this audit. Each file is mapped to the task that should modify it and a description of the required change. Files not listed here do not require changes for the signal

tracking contract work.

File	Task	Change Description
functions.ts	Task 2	Rewrite updateSignalTracking to call engine instead of trusting client. Remove evaluateTrackingPrice().
use-signal-monitor.ts	Task 6	Demote to price reporter only: send {trackingId, observedPrice, observedAt}. Remove all evaluation logic.
types.ts	Task 1.2	Remove invalidated from SignalStatus OR add DB migration. Update TERMINAL_STATUSES.
transition-engine.ts	Task 1.2	Add state/hitTp consistency validation. Possibly derive expected TP internally.
orchestrator.ts	Task 4	Align event names with engine vocabulary. Add per-TP event granularity.
context-engine.ts	Task 5	Update activeTrackings query: add tp1_hit, tp2_hit to status filter.
notification-hub.ts	Task 5	Update status filters to include tp1_hit, tp2_hit as active states.
reanalysis.ts	Task 2	May need engine consultation for invalidation logic.
signals.tsx	N/A	No change needed.
index.ts	N/A	Barrel file; may need re-exports after engine integration.
Migration SQL	Task 1.2	ALTER TYPE signal_status ADD VALUE 'invalidated' if approved.

The most impactful change is functions.ts (Task 2), which requires a near-complete rewrite of updateSignalTracking. The current function is a simple DB write. The new function must: (1) read the current tracking from DB, (2) construct a SignalTransitionRequest, (3) call evaluateSignalTransition(), (4) commit only if the engine allows, (5) derive hitTp from the resulting status, (6) handle idempotency and concurrency. This is the single most important code change in the entire task sequence.

14. P0 / P1 / P2 Classification

This section provides the definitive severity classification for all 9 identified issues. Each issue is assigned a priority level, a rationale, and a resolution target. P0 issues must be fixed before Task 2 begins. P1 issues must be fixed before live deployment. P2 issues can be deferred to the appropriate task.

14.1 P0 — Must Fix Before Task 2

#	Issue	Rationale
1	Connect engine to runtime path	Engine is dead code without this; all server validation depends on it
2	Remove client authority over status/hitTp	10 attack paths are open until this is fixed
3	Unify terminal state definitions	tp1_hit/tp2_hit conflict breaks multi-TP signals
4	Fix useSignalMonitor for tp1_hit/tp2_hit	Monitor stops too early, preventing TP2/SL detection
5	Add state/hitTp consistency validation	Forged hitTp shifts engine evaluation to wrong TP level
6	Decide invalidated semantics + DB migration if needed	Engine can produce invalidated but DB rejects it; blocks Task 2

14.2 P1 — Must Fix Before Live

#	Issue	Rationale
7	Document and enforce non-price transition policy	Cancel after partial TP has undefined business behavior
8	Implement observation timestamp contract	No serverReceivedAt, no market data timestamp, no ordering

14.3 P2 — Defer

#	Issue	Rationale
9	Unify event vocabulary (engine vs orchestrator)	Events are dead code currently; no functional impact until Task 4

15. Risks

This section identifies risks associated with implementing the recommended changes. Each risk has a mitigation strategy that should be considered during implementation planning.

#	Risk	Impact	Mitigation
1	Legacy DB rows with inconsistent status/hitTp	Engine may reject valid transitions on old data	Data audit + repair script before Task 2
2	Changing TERMINAL_STATUSES behavior	tp1_hit/tp2_hit signals become 'active' again for existing users	Feature flag or gradual rollout
3	evaluateTrackingPrice() parallel logic	If kept, it WILL diverge from engine	Must remove in Task 2, not refactor
4	reanalysis-cron status filter	Queries .in(['pending','active']) — misses tp1_hit/tp2_hit	Update filter in Task 2
5	MOXI context-engine status filter	Same miss as reanalysis-cron	Update in Task 5

Risk #1 is the most operationally dangerous. If the engine is connected and legacy rows have status=tp1_hit with hitTp=0 (inconsistent), the engine would derive expected hitTp=1 from status=tp1_hit, find inconsistency, and reject the transition. A data audit must scan all signal_tracking rows for state/hitTp inconsistencies and repair them before the engine goes live. Risk #2 affects user experience: signals that were 'finished' (in the old model) will become 'active' again and the monitor will start tracking them. For users who saw a TP1 notification and considered the signal done, this could be confusing.

16. Open Decisions

This section documents all decisions that are deferred beyond this audit. Each decision includes the current recommendation and the target task for resolution.

#	Decision	Recommendation	Target Task
1	Should hitTp be derived from status?	YES — derive from status on every write	Task 2

#	Decision	Recommendation	Target Task
2	Is invalidated persisted or event-only?	Defer — needs business analysis	Task 1.2
3	Cancel allowed after partial TP?	YES — with resolved_at recording partial result	Task 1.2
4	TP crossing policy: live vs OHLC?	One per observation for live; defer OHLC	Task 2 / future
5	Should engine validate state/hitTp?	YES — add as pre-check in engine entry	Task 1.2
6	Remove or refactor evaluateTrackingPrice?	REMOVE entirely — engine replaces it	Task 2

Decision #2 (invalidated semantics) is the most consequential open item. If invalidated is signal-level (the analysis was wrong, all trackings should stop), it requires a different implementation than if it is tracking-level (this particular tracking instance is invalid). If it is event-only (just emit an event, don't change DB status), it doesn't need the schema migration at all. The recommendation is to defer this to Task 1.2 with a dedicated business analysis session that includes product, engineering, and operations stakeholders. Decision #6 is straightforward: evaluateTrackingPrice() in functions.ts duplicates engine logic and will inevitably diverge if kept. It should be removed entirely in Task 2.

17. Proposed Task 2 Scope

This section defines the exact scope of Task 2 (Server Authority). Task 2 is the primary implementation task that resolves P0 issues #1-5 from Section 14. It rewrites the server-side signal update path to use the Transition Engine as the sole authority for status transitions.

17.1 Task 2 Deliverables

#	Deliverable	Details
1	Rewrite updateSignalTracking()	Accept {trackingId, observedPrice, observedAt}. Read DB state, call engine, commit if allowed.
2	Server function flow	Read current tracking → construct SignalTransitionRequest → call evaluateSignalTransition() → commit
3	Remove evaluateTrackingPrice()	Delete from functions.ts entirely. Engine replaces all price evaluation logic.
4	Add consistency validation	Engine pre-check: validate state/hitTp consistency on entry. Reject inconsistent requests.
5	Unify TERMINAL_STATUSES	Remove tp1_hit and tp2_hit from TERMINAL_STATUSES in types.ts.
6	Update useSignalMonitor	Continue monitoring for tp1_hit and tp2_hit states. Send {trackingId, observedPrice, observedAt} only.
7	Update status filters	reanalysis cron, MOXI context-engine: add tp1_hit and tp2_hit to active status filters.
8	Add idempotency protection	Deterministic key based on (trackingId, observedAt, observedPrice). Reject duplicates.
9	DO NOT change DB schema	That is Task 1.2 scope if needed.
10	DO NOT change event system	That is Task 4 scope.

17.2 Out of Scope for Task 2

- Database schema changes (Task 1.2)
- Event system unification (Task 4)
- MOXI notification-hub updates (Task 5)
- Client-side monitoring architecture changes (Task 6)
- Invalidated semantics decision (Task 1.2)
- OHLC multi-TP crossing policy (future decision)

Task 2 is the critical path item. All subsequent tasks depend on the server having proper authority over status transitions. The estimated complexity is moderate-to-high: the `updateSignalTracking` rewrite is the largest single change, but the remaining deliverables are relatively contained. The task should be approached incrementally: first add engine consultation (read-only mode, log but don't commit), then switch to engine-committed mode, then add idempotency and consistency validation. This incremental approach allows testing at each stage without breaking the existing runtime.

— End of Audit Report —